

How the AI Assistant & MCP Work

The email builder's AI, explained the easy way
Winaity Template Builder

A RESTAURANT-KITCHEN GUIDE

You order a dish. In your app, the "dish" is an **email**. Four characters work together in the kitchen to make it. Meet them one at a time, and the whole system clicks.

The whole system = a restaurant

In the kitchen	In your app	Its job
 The chef	vLLM (<code>gemma4-26b</code>)	Smart, speaks French, decides <i>what</i> to make — but has shaky hands .
 The kitchen tools	The tools (<code>addHeading</code> , <code>addButton</code> ...)	Each does <i>one</i> small job. The chef can only cook <i>using</i> these.
 The order-ticket system	MCP	The standard way the chef tells the tools what to do.
 The plate	The TemplateBuilder	Where the dish is assembled, one ingredient at a time.

Keep this picture in your head. Now let's meet each character slowly.

Character 1 — The chef 🍳 = vLLM

What it is: a private "ChatGPT" that runs on *your own* GPU, not OpenAI's. When your code needs the AI to think, it sends a message to this server and gets an answer back.

```
Your app:  "Make me a promo email for a gym"
           |
           v
         🍳 vLLM (gemma4-26b)
           |
           v
Your app:  gets the chef's answer
```

The ONE thing to remember about this chef:

He is **smart but clumsy**. He knows *what* a great email should say. But if you ask him to do 30 careful steps in a row, he trips, repeats himself, or drops things.

Everything unusual in your code exists to protect against this clumsiness. That one fact explains all the rest.

Character 2 — The tools 🔧

The chef **cannot touch the email directly**. He can't "write HTML." Instead, he has a row of buttons. Each does exactly one thing:

```
[ setTheme ]    → pick the colors of the whole email
[ addHeading ] → add a big title
[ addText ]     → add a paragraph
[ addButton ]   → add a clickable button
[ addImage ]    → add a picture
[ startHero ]   → add a big banner with a background photo
... about 32 buttons total ...
```

To build an email, the chef presses buttons **in order, top to bottom**:

```
Press setTheme("blue")
Press startHero("gym photo")
Press addHeading("Get Fit in 2026")
Press addButton("Join now")
Press ... footer ...
```

Why buttons instead of free writing? Because he's clumsy. Raw HTML would come out broken. But a *button* can only produce a correct result — press **addButton** and you always get a valid button. **The email can never come out broken.** That is the entire reason tools exist.

Each button also carries a little **label describing itself**, e.g. "addButton — adds a clickable call-to-action button. Needs: text, url." Remember that label idea — it matters next.

Character 3 — MCP 📋 = how the chef presses the buttons

Where are these buttons? Not next to the chef — in a separate room, a little **machine** in your app at this address:

```
/builder/mcp
```

MCP is just the agreed language for talking to that machine over the internet — like a standardized order ticket. Any chef who "speaks MCP" can walk up and use the machine.

Step A — "What buttons do you have?"

```
Your app —"what tools?"—> /builder/mcp  
Your app ←—"here: setTheme (picks colors), addHeading (adds title), ... "
```

This is the clever bit. The chef learns the buttons *from the machine itself*, right at that moment. So if you add a new button to the machine later, the chef can use it instantly — **you don't have to re-teach him**.

Step B — "Press this button"

```
Chef —"press addHeading with text='Get Fit'"—> machine  
machine: (adds the heading to the plate) "ok, done ✅"
```

One order = one plate

Every order gets its **own machine + own plate**, tagged with a session ID. Two users building at the same time never mix ingredients. When the order's done, the plate is served and cleared.

```
Order #abc → machine #abc → plate #abc (this user's email)  
Order #xyz → machine #xyz → plate #xyz (another user's email)
```

Character 4 — The plate 🍽️ = TemplateBuilder

The plate is the email being built up. Every button press adds one ingredient:

```
Empty plate: (nothing)  
  
after setTheme:    { colors: blue }  
after startHero:   { colors: blue, banner: [photo] }  
after addHeading:  { colors: blue, banner: [photo], title: "Get Fit" }  
after addButton:   { ... + button: "Join now" }
```

At the very end, your app presses one special button — **getTemplate** — meaning **"give me the finished plate."** That finished plate (a big JSON object) is the email shown on screen.

The full order, start to finish

User types: "make me a gym promo email."

1. User's request arrives at your app (/api/ai/chat)
2. App walks to the machine and opens a new order (openBuildSession)
3. App asks "what buttons exist?" (listTools)
4. App tells the chef: "here are your buttons, now build a gym email" (streamText + vLLM)
5. Chef presses buttons one by one: (tool calls)
setTheme → startHero → addHeading → addButton → ... → footer
(each press adds to the plate 🍳)
6. App says "give me the finished plate" (getTemplate)
7. App cleans it up + shows the email to the user (done ✅)

That's the **MCP path**. Now the twist that confuses most people.

The twist: your app usually AVOIDS the clumsy chef

Remember — the chef is **clumsy at long sequences**. Pressing 30 buttons in a row is exactly where he trips. So your team invented a **smarter way**, the **Planner**.

Instead of asking the chef to press 30 buttons himself, you ask him for a simple **written recipe** — just once:

- ❌ OLD way (clumsy):
Chef presses 30 buttons in a row → often trips → broken email
- ✅ NEW way (safe):
Chef writes ONE recipe card → YOUR CODE presses the buttons perfectly

The recipe card is a structured list called a **DesignSpec**:

```
{
  colors: blue,
  designSystem: "bold",
  sections: [
    { kind: "hero", heading: "Get Fit in 2026", cta: "Join now" },
    { kind: "features", items: ["Free trial", "24/7 access"] },
    { kind: "pricing", plans: [ ... ] },
    { kind: "footer" }
  ]
}
```

The chef is **great** at writing this recipe (one clean answer, not 30 shaky steps). Then **your code** — which never trips — reads the recipe and presses all the buttons flawlessly.

The chef decides *what* to say. Your code handles *how it's built*. This is why the Planner is the *preferred* path, and MCP button-pressing is the *backup*.

One more layer: skip the chef entirely when you can

For simple edits, your app doesn't even wake the chef:

```
User says "make it dark mode"    → your code flips colors.  No chef.  ⚡ instant
User says "delete everything"    → your code clears the plate. No chef. ⚡ instant
User says "it's unreadable"     → your code fixes colors.  No chef.  ⚡ instant
User says something creative    → NOW wake the chef.
```

Same reason: the chef is clumsy and slow, so you only call him when you truly need his *brain* (understanding language, writing copy). Anything a plain rule can do, your code does itself.

The final summary (fridge-magnet version)

- 🤖 **vLLM** = your private AI chef. Smart, French-speaking, but clumsy with long step-by-step work.
- 🛠️ **Tools** = buttons that each build one email piece. The chef can *only* build by pressing buttons, so the email is never broken.
- 📋 **MCP** = the standard "order ticket" system for talking to the button-machine at `/builder/mcp`. The chef even learns the buttons from the machine on the spot.
- 🍽️ **TemplateBuilder** = the plate the email is assembled on, one press at a time. `getTemplate` serves it.
- 🗨️ **Planner** = the smart shortcut: chef writes *one recipe card*, your reliable code presses the buttons. Preferred, because it dodges the chef's clumsiness.
- ⚡ **Instant paths** = simple requests (dark mode, delete all) skip the chef completely.

Where each part lives in the code

Part	File
The chef (vLLM connection)	<code>frontend/lib/ai/provider.ts</code>
The button-machine (MCP server)	<code>frontend/app/builder/mcp/route.ts</code> + <code>lib/ai/mcp/build-server.ts</code>
Talking to the machine (MCP client)	<code>frontend/lib/ai/mcp/client.ts</code>
One build turn	<code>frontend/lib/ai/mcp/generate.ts</code>
The buttons themselves (tools)	<code>frontend/lib/ai/tools.ts</code>
The recipe shortcut (Planner)	<code>frontend/lib/ai/planner.ts</code>
The prompts (chef's instructions)	<code>frontend/lib/ai/system-prompt.ts</code>
The traffic controller (routing)	<code>frontend/app/api/ai/chat/route.ts</code>